

Data Analysis with Python 2024–2025

Parte 5: Machine Learning (Regressão Linear)

Conteúdo Teórico

Tradução e Adaptação: University of Helsinki — MOOC.fi

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024-2025*, oferecido pela **The University of Helsinki MOOC Center**.

O que você vai aprender nesta página

- Compreender o conceito de regressão linear.

Conteúdo

1 Regressão Linear

A análise de regressão tenta explicar as relações entre variáveis. Uma dessas variáveis, chamada de variável dependente, é o que queremos "explicar" usando uma ou mais variáveis explicativas (ou independentes). Na regressão linear, assumimos que a variável dependente pode ser, aproximadamente, expressa como uma combinação linear das variáveis explicativas.

Como um exemplo simples, podemos ter a variável dependente "altura" e uma variável explicativa "idade". A idade de uma pessoa pode explicar muito bem a sua altura, e essa relação é aproximadamente linear para crianças (idades entre 1 e 16 anos). Outra forma de pensar sobre a regressão é ajustar uma curva aos pontos de dados observados. Se temos apenas uma variável explicativa, isso é fácil de visualizar, como veremos a seguir.

Podemos aplicar a regressão linear facilmente com o pacote `scikit-learn`. Vamos analisar alguns exemplos.

Primeiro, fazemos as importações padrão.

```
import numpy as np
import matplotlib.pyplot as plt
import sklearn # Isso importa a biblioteca scikit-learn
```

Em seguida, criamos alguns dados com aproximadamente a relação $y = 2x + 1$, com erros normalmente distribuídos.

```
np.random.seed(0)
n = 20 # Numero de pontos de dados
x = np.linspace(0, 10, n)
y = x * 2 + 1 + 1 * np.random.randn(n) # Desvio padrao 1
print(x)
print(y)
```

O próximo passo é importar a classe `LinearRegression`.

```
from sklearn.linear_model import LinearRegression
```

Agora podemos ajustar uma linha através dos pontos de dados (x, y) :

```
model = LinearRegression(fit_intercept=True)
model.fit(x[:, np.newaxis], y)
xfit = np.linspace(0, 10, 100)
yfit = model.predict(xfit[:, np.newaxis])
plt.plot(xfit, yfit, color="black")
plt.plot(x, y, 'o')
# O código a seguir desenhara tantos segmentos de reta quantas forem as
# colunas nas matrizes x e y
plt.plot(np.vstack((x, x)), np.vstack([y, model.predict(x[:, np.newaxis])]), color="red")
```

A regressão linear tenta minimizar a soma dos erros quadráticos:

$$\sum_i (y[i] - \hat{y}[i])^2$$

que equivale à soma dos comprimentos ao quadrado dos segmentos de reta (que no exemplo estariam em vermelho) no gráfico traçado. Os valores estimados $\hat{y}[i]$ são denotados por `yfit[i]` no código acima.

```
print("Parameters:", model.coef_, model.intercept_)
print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)
```

Neste caso, o *coefficient* (coeficiente) é a inclinação da linha ajustada, e o *intercept* (intercepto) é o ponto onde a linha ajustada intercepta o eixo y.

Os parâmetros estimados pelo algoritmo de regressão costumam ser bem próximos dos parâmetros que geraram os dados (neste caso, coeficiente 2 e intercepto 1).

2 Múltiplas Features (Variáveis Explicativas)

O exemplo anterior tinha apenas uma variável explicativa. Algumas vezes, isso é chamado de regressão linear simples. O próximo exemplo ilustra uma regressão mais complexa com múltiplas variáveis explicativas.

```
sample1 = np.array([1, 2, 3]) # As tres variaveis explicativas tem
                               valores 1, 2 e 3, respectivamente
sample2 = np.array([4, 5, 6]) # Outro exemplo de valores das variaveis
                               explicativas
sample3 = np.array([7, 8, 10]) # ...

# Para os valores 1, 2 e 3 das variaveis explicativas, o valor y=15 foi
  observado, e assim por diante.
y = np.array([15, 39, 66]) + np.random.randn(3)
```

Vamos tentar ajustar um modelo linear a esses pontos:

```
model2 = LinearRegression(fit_intercept=False)
x = np.vstack([sample1, sample2, sample3])
model2.fit(x, y)
print(model2.coef_, model2.intercept_)
```

Vamos imprimir os vários componentes envolvidos.

```
b = model2.coef_[0, np.newaxis]
print("x:\n", x)
print("b:\n", b)
print("y:\n", y[:, np.newaxis])
print("product:\n", np.matmul(x, b))
```

3 Regressão Polinomial

Pode ser uma surpresa que é possível ajustar uma curva polinomial aos pontos de dados utilizando regressão linear. O truque é adicionar novas variáveis explicativas ao modelo. Abaixo, temos uma única feature x com valores y associados, dados por um polinômio de terceiro grau, com algum ruído (gaussiano) adicionado. Torna-se claro que não podemos explicar os dados bem com uma função linear, então adicionamos duas novas features: x^2 e x^3 . O modelo linear encontrará os coeficientes para essas variáveis.

```
x = np.linspace(-50, 150, 50)
y = 0.15 * x**3 - 20 * x**2 + 5 * x - 4 + 5000 * np.random.randn(50)
plt.scatter(x, y, color="black")

model_linear = LinearRegression(fit_intercept=True)
model_squared = LinearRegression(fit_intercept=True)
model_cubic = LinearRegression(fit_intercept=True)

x2 = x**2
x3 = x**3
```

```

model_linear.fit(np.vstack([x]).T, y)
model_squared.fit(np.vstack([x, x2]).T, y)
model_cubic.fit(np.vstack([x, x2, x3]).T, y)

xf = np.linspace(-50, 150, 50)
yf_linear = model_linear.predict(np.vstack([xf]).T)
yf_squared = model_squared.predict(np.vstack([xf, xf**2]).T)
yf_cubic = model_cubic.predict(np.vstack([xf, xf**2, xf**3]).T)

plt.plot(xf, yf_linear, label="linear")
plt.plot(xf, yf_squared, label="squared")
plt.plot(xf, yf_cubic, label="cubic")
plt.legend()

print("Coefficients:", model_cubic.coef_)
print("Intercept:", model_cubic.intercept_)

```

Variáveis lineares e ao quadrado não são suficientes para explicar os dados, mas a regressão linear consegue ajustar muito bem uma curva polinomial aos pontos de dados quando variáveis cúbicas são incluídas.

4 Resumo (Semana 5)

- `pd.concat` e `pd.merge` podem combinar dois DataFrames, mas a forma de combinação difere. A função `pd.concat` concatena com base nos índices, enquanto `pd.merge` combina com base no conteúdo de variáveis comuns.
- A opção `join="outer"` no `pd.concat` pode criar valores ausentes, mas `join="inner"` não. O primeiro retorna a união dos índices, e o segundo, a intersecção.
- No `pd.concat`, índices sobrepostos podem: causar um erro, renumeração de índices, ou criar índices hierárquicos.
- O "merging" (mesclagem) pode unir elementos de um-para-um, um-para-muitos, ou muitos-para-muitos.
- No agrupamento (grouping), um DataFrame é dividido em DataFrames menores. As principais operações nestes grupos são: `aggregate`, `filter`, `transform` (mantém o formato) e `apply`.
- Series indexadas pelo tempo são chamadas de séries temporais (time series).
- A regressão linear pode ser usada para descobrir relacionamentos lineares entre variáveis: pode ter mais de uma feature (variável explicativa) e ajustar polinômios ainda é considerado regressão linear.

Créditos: Este conteúdo foi originalmente criado pela *The University of Helsinki MOOC Center* para o curso *Data Analysis with Python*.